

how-to-experimental.md

# HOW-TO – Ambiente de Simulação SDN + QUIC-sim + Transferência de Arquivo

## Redes e Computação em Nuvem — Trabalho Final

---

### Premissas

Este HOW-TO assume:

1. A VM fornecida **já possui Ryu instalado e funcional**.
  2. A VM está **atualizada** e pronta para uso.
  3. Os arquivos fornecidos pelo professor estão disponíveis em `~/sdn-lab/`:
    - `simple_switch_final.py`
    - `udp_server_final.py`
    - `udp_client_final.py`
    - `topo_malha.py`
  4. Todo o experimento deve usar **processos rodando em background (&)** dentro do Mininet.
- 

## 1. Preparação dos Arquivos

Crie (ou confirme) o diretório de trabalho:

```
mkdir -p ~/sdn-lab  
cd ~/sdn-lab
```

Coloque dentro dele os arquivos fornecidos:

```
simple_switch_final.py  
udp_server_final.py
```

```
udp_client_final.py
topo_malha.py
arquivo_teste.txt      # arquivo a ser enviado no experimento
```

---

## 2. Executar o Controlador SDN (Ryu)

Abra um terminal separado:

```
source venv-ryu/bin/activate
cd ~/sdn-lab
ryu-manager simple_switch_final.py
```

Não feche este terminal. Ele mostrará logs importantes:

- PACKET\_IN
  - fluxos instalados
  - decisões SDN
- 

## 3. Subir a Topologia Mesh com 4 Roteadores

Em outro terminal:

```
sudo mn --custom topo_malha.py --topo mesh4 \
        --controller=remote --switch ovsk --mac
```

Você verá os elementos:

h1, h2, s1, s2, s3, s4

Verifique os IPs dos hosts:

```
mininet> h1 ip a
```

```
mininet> h2 ip a
```

Normalmente:

- h1 → 10.0.0.1
- h2 → 10.0.0.2

---

## **\*\*4. Executando o QUIC-sim \*\***

### **4.1. Rodar o servidor QUIC-sim em background (no h1)**

```
mininet> h1 python3 udp_quic_server.py &
```

Esse comando:

- executa servidor em background
- mantém o Mininet utilizável
- permite rodar o cliente simultaneamente

Você verá algo como:

```
[1] 1234
```

Significa: job 1, PID 1234.

---

### **4.2. Rodar o cliente QUIC-sim no h2**

```
mininet> h2 python3 udp_client_final.py
```

O cliente realizará:

- handshake
- envio de mensagens
- envio do arquivo (em chunks, com ACK por chunk)
- retransmissão em caso de timeout
- medição de RTT por chunk

A saída incluirá:

```
[SEND] seq=1002 tentativa=1  
[ACK OK] seq=1002 rtt=0.154s  
[TIMEOUT] seq=1007 tentativa=2  
...
```

---

## 4.3. Verificar processos em execução

```
mininet> h1 ps -ef | grep python
```

---

## 4.4. Encerrar o servidor QUIC-sim

```
mininet> h1 kill %python3
```

---

# 5. Testando a Manipulação SDN

A manipulação SDN ocorre **exclusivamente** no arquivo `simple_switch_final.py`.

O aluno deve implementar as políticas detalhadas no **enunciado oficial**:

- baseline
  - rota superior forçada
  - rota inferior forçada
  - ECMP-sim (balanceamento)
  - falha de link + reroteamento
  - política de segurança
- 

## 5.1. Após alterar o controlador:

No terminal do Ryu, pressione:

```
Ctrl + C
```

---

Reinicie o controlador:

```
ryu-manager simple_switch_final.py
```

---

## 5.2. Limpar e reiniciar a topologia (quando necessário)

No Mininet:

```
mininet> exit  
sudo mn -c
```

Reiniciar topologia:

```
sudo mn --custom topo_malha.py --topo mesh4 \  
--controller=remote --switch ovsk --mac
```

---

# 6. Verificando Fluxos Instalados no OVS

Para cada switch:

```
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s1  
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s2  
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s3  
mininet> sh ovs-ofctl -O OpenFlow13 dump-flows s4
```

Os dumps ajudam a comprovar:

- instalação correta das rotas
- prioridade das regras
- manipulação de caminhos
- políticas de bloqueio/prioridade

## 7. Coleta de Métricas — O que registrar

Os dados abaixo são **obrigatórios** para cada experimento:

---

### 7.1. RTT por chunk

Registrar saída do cliente:

```
[ACK OK] seq=1002, rtt=0.154s
```

---

### 7.2. Retransmissões por chunk

Detectado quando o cliente imprime:

```
[TIMEOUT] seq=1007 tentativa=1
```

---

### 7.3. Throughput efetivo

Calcular:

$$\text{tamanho\_total\_arquivo} / \text{tempo\_total\_envio}$$

---

### 7.4. Integridade do arquivo

No cliente e servidor:

```
md5sum arquivo_teste.txt
md5sum recebido.bin
```

Hashes devem coincidir.

---

### 7.5. Logs do SDN

Copiar trechos relevantes do terminal do Ryu:

FLOW\_MOD  
PACKET\_IN  
decision: out\_port=...

---

## 7.6. Fluxos OpenFlow instalados

Inserir no relatório:

- prints dos dumps
  - breve explicação de quais regras aparecem e por quê
- 

## 8. Falha de Link e Recuperação

Simule falha:

```
mininet> link s1 s2 down
```

Observe:

- servidor recebendo menos ACK
- cliente retransmitindo mais chunks
- queda temporária no throughput

O controlador deve redirecionar QUIC-sim via:

**s1 → s3 → s4**

Restaurar link:

```
mininet> link s1 s2 up
```

Registrar o comportamento:

- antes
  - durante
  - depois
-



## 9. Limpar o Ambiente

Após finalizar:

```
mininet> exit  
sudo mn -c
```

Encerrar o controlador SDN com Ctrl+C.

---

## 10. Ajustes no cliente (udp\_client\_final.py)

### 10.1. Novos imports

Adicione no topo:

```
import base64  
import os
```

---

### 10.2. Configurar o nome do arquivo a ser enviado

Perto das outras constantes (SERVER\_IP, SERVER\_PORT, etc.), adicione:

```
FILE_TO_SEND = "arquivo_teste.txt" # ajuste para o arquivo que você quiser  
CHUNK_SIZE = 1024                  # bytes por chunk
```

---

### 10.3. Função para quebrar o arquivo em chunks

Logo depois das funções existentes, adicione:

```
def load_file_chunks(filename, chunk_size=1024):  
    """Lê o arquivo em binário e retorna lista de chunks (bytes)."""  
    with open(filename, "rb") as f:  
        data = f.read()  
  
    chunks = [
```



```
    data[i:i + chunk_size]
    for i in range(0, len(data), chunk_size)
]
return chunks
```

## 10.4. Mandar o arquivo depois do handshake (e opcionalmente depois dos pacotes data)

No `main()`, depois do handshake (e depois dos pacotes data se você manteve), adicione algo assim:

```
# --- ENVIO DE ARQUIVO (NOVO) ---
if not os.path.exists(FILE_TO_SEND):
    print(f"[CLIENTE] Arquivo {FILE_TO_SEND} não encontrado, pulando envio de arquivo.")
    return

chunks = load_file_chunks(FILE_TO_SEND, CHUNK_SIZE)
total = len(chunks)
print(f"[CLIENTE] Enviando arquivo {FILE_TO_SEND} em {total} chunks de até {CHUNK_SIZE} bytes.")

seq_base = 1000 # só para separar da sequência dos outros pacotes

for i, raw_chunk in enumerate(chunks):
    seq = seq_base + i
    b64_chunk = base64.b64encode(raw_chunk).decode()

    pkt = {
        "type": "file_chunk",
        "seq": seq,
        "total": total,
        "filename": os.path.basename(FILE_TO_SEND),
        "data": b64_chunk,
    }

    ok, rtt = send_and_wait_ack(pkt, server_addr)
    if not ok:
        print(f"[CLIENTE] Falha ao enviar chunk seq={seq}. Encerrando transmissão de arquivo.")
        break

print("[CLIENTE] Fim da transmissão do arquivo.")
```

A função `send_and_wait_ack` que usamos antes (para handshake e data) **já funciona** para os `file_chunk`, porque ela só verifica o `seq` no ACK, não o `type`. Então não precisa mexer nela.

## 11. Ajustes no servidor (`udp_server_final.py`)

### 11.1. Novos imports

No topo:

```
import base64
from collections import defaultdict
```

### 11.2. Estruturas para guardar os chunks

Antes do loop principal, adicione:

```
# Armazena chunks por arquivo: filename -> { "total": int, "chunks": {seq: bytes} }
files_state = defaultdict(lambda: {"total": None, "chunks": {}})
```

### 11.3. Função para reconstruir o arquivo

Adicione antes do `while True::`

```
def rebuild_file(filename, file_info):
    """Reconstrói o arquivo a partir dos chunks recebidos."""
    chunks_dict = file_info["chunks"]
    total = file_info["total"]

    # Ordena pelos seq
    ordered_seqs = sorted(chunks_dict.keys())
    if len(ordered_seqs) != total:
        print(f"[SERVIDOR] Aviso: número de chunks ({len(ordered_seqs)}) diferente de total ({total})")

    output_name = f"recebido_{filename}"
    with open(output_name, "wb") as f:
        for seq in ordered_seqs:
```

```
f.write(chunks_dict[seq])

print(f"[SERVIDOR] Arquivo reconstruído como {output_name}")
```

## 11.4. Tratamento específico para pacotes file\_chunk

Dentro do loop principal `while True`: onde você já faz:

```
packet = json.loads(data.decode())
ptype = packet.get("type", "unknown")
seq = packet.get("seq", "?")
```

Adicione um bloco **ANTES** do resto do tratamento (ou pelo menos antes do `response = {... ack ...}` genérico):

```
# --- NOVO: tratamento de file_chunk ---
if ptype == "file_chunk":
    filename = packet.get("filename", "arquivo.bin")
    total = packet["total"]
    b64_data = packet["data"]

    chunk_bytes = base64.b64decode(b64_data)

    file_info = files_state[filename]

    # define total na primeira vez
    if file_info["total"] is None:
        file_info["total"] = total

    file_info["chunks"][seq] = chunk_bytes

    print(f"[SERVIDOR] Recebido chunk seq={seq} do arquivo {filename} "
          f"({len(file_info['chunks'])}/{file_info['total']})")

    # Envia ACK específico
    ack = {
        "type": "ack_chunk",
        "seq": seq
    }
    sock.sendto(json.dumps(ack).encode(), addr)

    # Se completou todos os chunks, reconstrói
```

```
if len(file_info["chunks"]) == file_info["total"]:  
    rebuild_file(filename, file_info)  
  
# Já tratamos este tipo de pacote, volta pro início do loop  
continue
```

Importante: esse `continue` garante que o restante da lógica (handshake/data) não vai tentar processar o `file_chunk`.

O resto do código (handshake, data, perdas simuladas, etc.) pode ficar como estava.

---